



Estructuras de Datos y Algoritmos II

LORENA RECALDE Ph.D.

Escuela Politécnica Nacional

2026-A

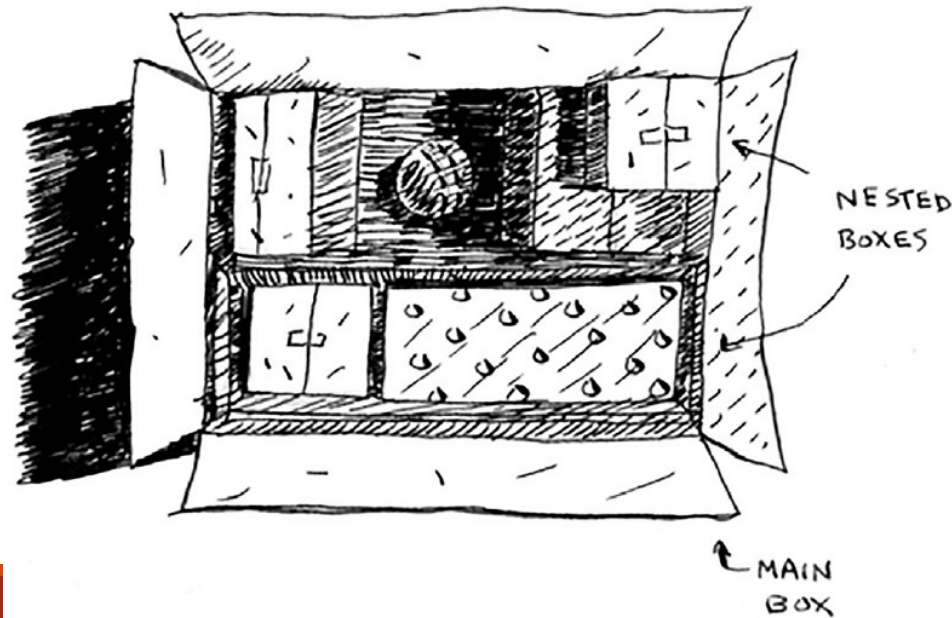
Revisión: Recursividad

Introducción

- La recursividad es una forma elegante de resolver problemas, pero es *controversial*. La gente lo ama o lo odia, o lo odia hasta que aprende a amarlo unos años más tarde.
- Para facilitarle las cosas, hay algunos consejos:
 - ✓ Busque ejemplos de código. **Ejecute el código usted mismo** para ver cómo funciona.
 - ✓ Al menos una vez, realice una función recursiva con **lápiz y papel**. Caminar a través de una función recursiva le enseñará cómo funciona.
- Usted puede generar *pseudocódigo* a manera de una descripción de alto nivel del problema que está intentando resolver.

Recursividad

- Supongamos que buscas la llave de una habitación cerrada.
- Y la llave probablemente está en esta caja.



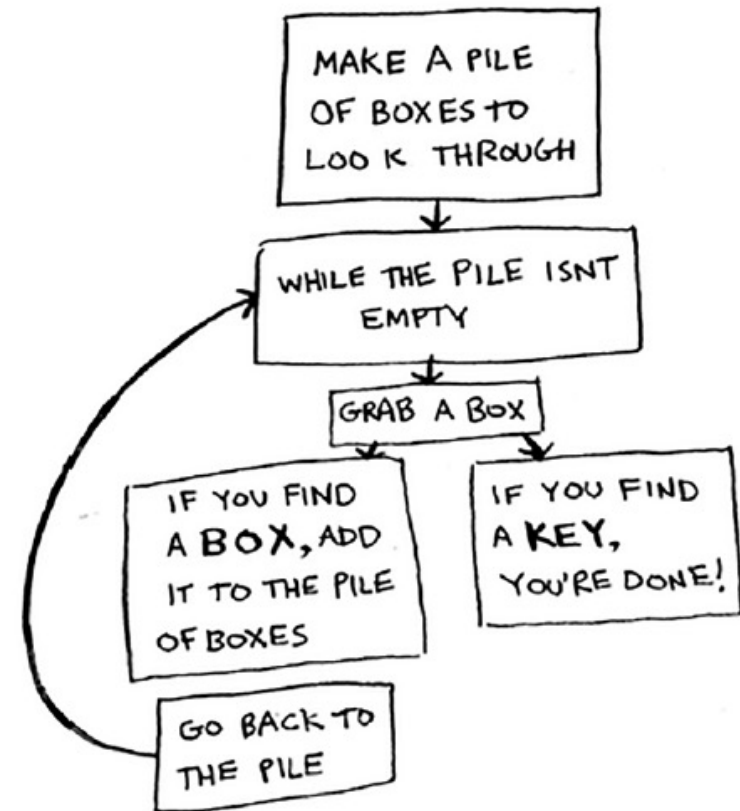
Recursividad

- Esta caja contiene más cajas, con **más cajas dentro de esas cajas**.
- La llave está en una caja en alguna parte.
- ¿Cuál es tu algoritmo para buscar la llave?

Recursividad

•Aquí hay un **primer enfoque**.

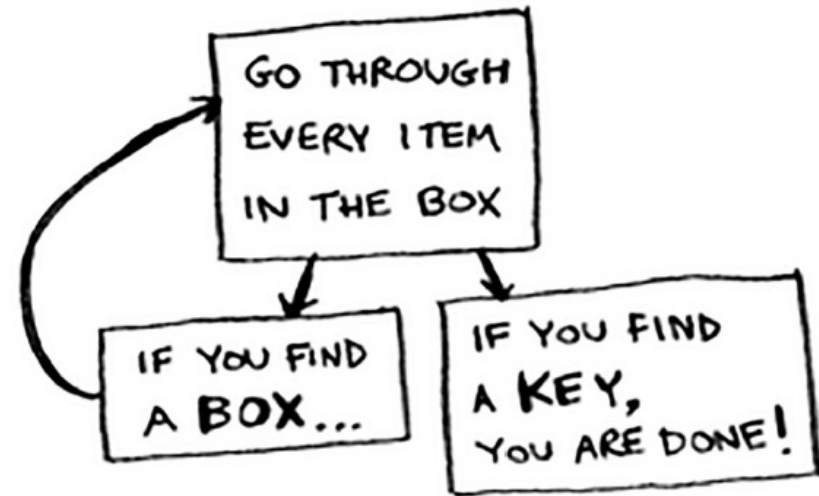
1. Haz una pila de un montón de cajas para mirar.
2. Toma una caja y mira en ella.
3. Si encuentras una caja, agrégala a la pila para verla luego.
4. Si encuentras una llave, ¡ya está!
5. Repite.



Recursividad

- Aquí hay un **segundo enfoque**.

1. Mira a través de la caja.
2. Si encuentras una caja, ve al paso 1.
3. Si encuentras una llave, ¡ya está!



Recursividad

- ¿Qué enfoque te parece más fácil? El primer enfoque utiliza un bucle *while*.
- Mientras la pila no está vacía, toma una caja y mira a través de ella:

```
def look_for_key(main_box):  
    pile = main_box.make_a_pile_to_look_through()  
    while pile is not empty:  
        box = pile.grab_a_box()  
        for item in box:  
            if item.is_a_box():  
                pile.append(item)  
            elif item.is_a_key():  
                print "found the key!"
```


Recursividad

- La segunda forma usa la recursividad. **La recursión es donde una función se llama a sí misma.** Aquí está la segunda forma en pseudocódigo:

```
def look_for_key(box):  
    for item in box:  
        if item.is_a_box():  
            look_for_key(item) ←..... Recursion!  
        elif item.is_a_key():  
            print "found the key!"
```

Recursividad

- Ambos enfoques logran lo mismo, *pero el segundo enfoque parece ser más claro.*
- *No hay beneficio de rendimiento* al usar la recursividad; de hecho, los bucles a veces son mejores para el **rendimiento**.
- Citando a Leigh Caldwell: “Los bucles pueden lograr una ganancia de rendimiento para su programa. La recursión puede lograr una ganancia de rendimiento para su programador. ¡Elija cuál es más importante en su situación!”
- **Muchos algoritmos importantes utilizan la recursividad, por lo que es importante comprender cómo trabaja.**

Base case and recursive case

- Debido a que una función recursiva se llama a sí misma, es fácil escribir una función incorrectamente que termina en un **bucle infinito**.
- Por ejemplo, suponga que desea escribir una función que imprima una cuenta regresiva, como esta:

> 3 ... 2 ... 1

- Puedes escribirlo recursivamente, así:

```
def countdown(i):  
    print i  
    countdown(i-1)
```



- Escriba este código y ejecútelo. Notará un problema: ¡esta función se ejecutará para siempre!

> 3 ... 2 ... 1 ... 0 ... -1 ... -2 ...

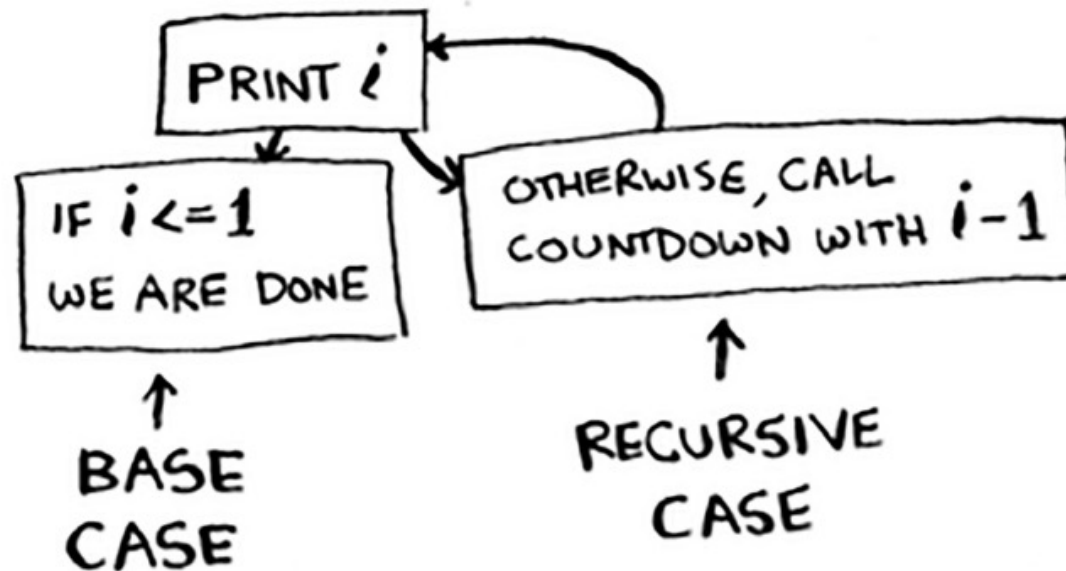
Base case and recursive case

- Cuando escribe una función recursiva, debe decirle cuándo dejar de recurrir.
- Es por eso que cada función recursiva tiene dos partes: el **caso base** y el **caso recursivo**.
- El caso recursivo es cuando la función se llama a sí misma.
- El caso base es cuando la función no se vuelve a llamar a sí misma ... por lo que *no entra en un bucle infinito*.
- Agreguemos un caso base a la función de cuenta regresiva:

```
def countdown(i):  
    print i  
    if i <= 0: <----- Base case  
        return  
    else: <----- Recursive case  
        countdown(i-1)
```

Base case and recursive case

- Ahora la función se ejecuta como se esperaba. Es algo parecido a esto.



Pilas

La Pila



- La **pila de llamadas** es un concepto importante en la programación general, y cuando se utiliza la recursividad.
- Supongamos que está planificando una barbacoa. Mantiene una lista de tareas para la barbacoa, en forma de una **pila de notas adhesivas**.
- ¿Recuerda el trabajo de arrays y listas enlazadas? Puede agregar elementos de tareas en cualquier lugar de la lista/arreglo o eliminar elementos de cualquier posición. **La pila de notas adhesivas es mucho más simple:**

- Cuando inserta un elemento, se agrega a la parte superior de la pila.
- Cuando lee un elemento, solo lee el elemento superior, y el elemento superior es el que se elimina de la pila.

La Pila

- Por lo tanto, su lista de tareas solo tiene dos acciones: **push** (insertar) y **pop** (eliminar y leer).



PUSH
(ADD A NEW ITEM
TO THE TOP)



POP
(REMOVE THE TOPMOST
ITEM AND READ IT)

La Pila

- Veamos la lista de tareas en acción.



- Esta estructura de datos se llama **pila**.
- La pila es una estructura de datos simple.

La Pila de Llamadas

La pila de llamadas

- Su computadora usa una pila *internamente* llamada **pila de llamadas**.
- Aquí hay una función simple:

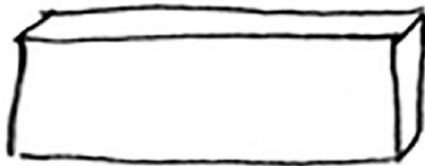
```
def greet(name):  
    print "hello, " + name + "!"  
    greet2(name)  
    print "getting ready to say bye..."  
    bye()
```

- Esta función te saluda y
- luego llama a otras dos funciones

```
def greet2(name):  
    print "how are you, " + name + "?"  
  
def bye():  
    print "ok bye!"
```

La pila de llamadas

- Veamos **qué sucede cuando llamas a una función.**
- Supongamos que llamas greet ("maggie").
- Primero, su computadora asigna una caja de memoria para esa llamada de función.



- Ahora usemos la memoria. El nombre de la variable se establece en "maggie". Eso necesita ser guardado en la memoria.



La pila de llamadas

```
def greet(name):  
    print "hello, " + name + !"  
    greet2(name)  
    print "getting ready to say bye..."  
    bye()
```

- Cada vez que realiza una llamada de función, su computadora guarda los valores para todas las variables para esa llamada en la memoria de esta manera. A continuación, imprime **hello, maggie!**
- Entonces llamas a **`greet2("maggie")`**.
- Nuevamente, *su computadora asigna una caja de memoria para esta llamada de función.*



La pila de llamadas

```
def greet2(name):  
    print "how are you, " + name + "?"
```

- Su computadora está usando una **pila** para estas cajas. La segunda caja se agrega encima de la primera. Imprimen **how are you, maggie?** Luego se hace el return de la llamada a la función. Cuando esto sucede, la caja en la parte superior de la pila sale.



La pila de llamadas

```
def greet(name):  
    print "hello, " + name + "  
    greet2(name)  
    print "getting ready to say bye..."  
    bye()
```

- Ahora la caja superior de la pila es para la función greet, lo que significa que **regresó a la función de saludo**.
- Cuando llamé a la función greet2, la función greet se completó **parcialmente**. Esta es la idea: *cuando llama a una función desde otra función, la función de llamada se detiene en un estado parcialmente completado*.
- **Todos los valores de las variables para esa función todavía se almacenan** en la memoria. Ahora que ha terminado con la función greet2, ha vuelto a la función greet y retomó donde lo dejó.
- Primero imprime **getting ready to say bye....** Llama a la función bye.

La pila de llamadas

```
def bye():  
    print "ok bye!"
```

- Se agrega un cuadro para esa función en la parte superior de la pila.



- Entonces imprimes **ok bye!** y se hace el retorno de la llamada a la función. Y has vuelto a la función *greet*.



La pila de llamadas

```
def greet(name):  
    print "hello, " + name + "  
    greet2(name)  
    print "getting ready to say bye..."  
    bye()
```

- No hay nada más que hacer, así que también **se hace el retorno** de la función de *greet*.
- Esta pila, utilizada para **guardar las variables para múltiples funciones**, se denomina *pila de llamadas*.

La pila de llamadas

EXERCISE

3.1 Suppose I show you a call stack like this.



What information can you give me, just based on this call stack?

La pila de llamadas

Answer: Here are some things you could tell me:

- The `greet` function is called first, with `name = maggie`.
- Then the `greet` function calls the `greet2` function, with `name = maggie`.
- At this point, the `greet` function is in an incomplete, suspended state.
- The current function call is the `greet2` function.
- After this function call completes, the `greet` function will resume.



Ahora veamos la pila de llamadas en acción con una función recursiva.

La pila de llamadas con recursividad

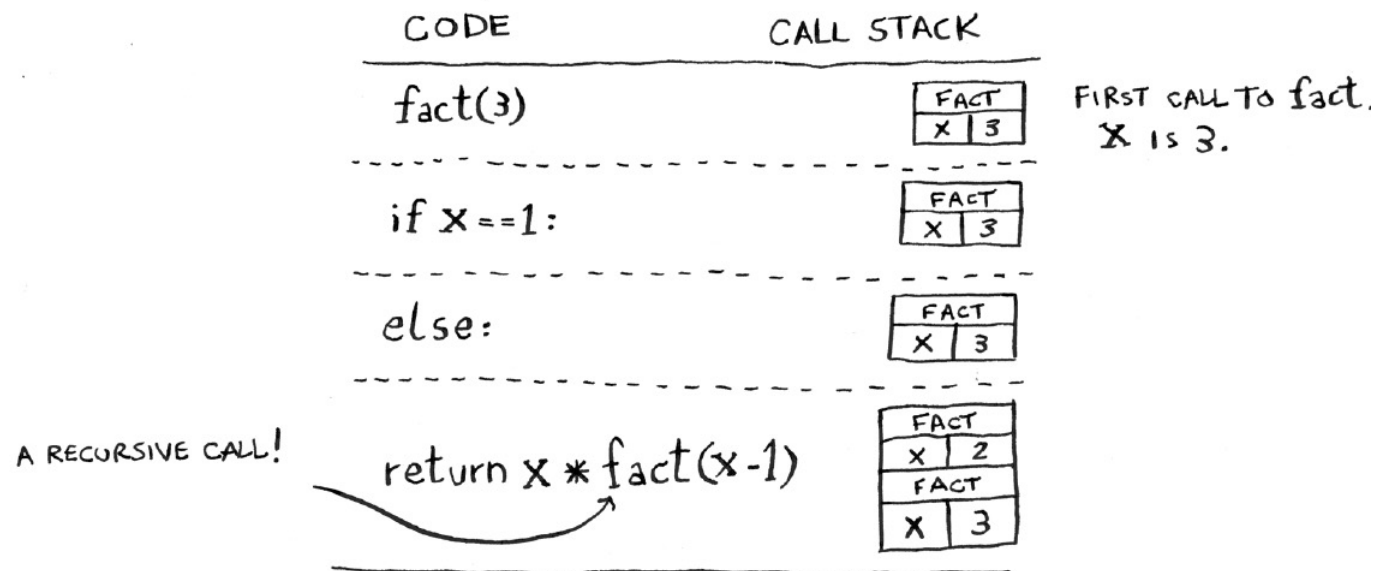
- ¡Las funciones recursivas también usan la pila de llamadas!
- Veamos esto con la función factorial. **factorial(5)** se escribe como 5!, y se define así: $5! = 5 * 4 * 3 * 2 * 1$.
- Del mismo modo, **factorial(4)** es $4 * 3 * 2 * 1$.
- Aquí hay una función recursiva para calcular el factorial de un número:

```
def fact(x):  
    if x == 1:  
        return 1  
    else:  
        return x * fact(x-1)
```

La pila de llamadas con recursividad

```
def fact(x):  
    if x == 1:  
        return 1  
    else:  
        return x * fact(x-1)
```

- Ahora llamas a la función **fact(3)**. Veamos esta llamada línea por línea y veamos cómo cambia la pila.
- Recuerde, la caja superior de la pila le dice qué llamada a fact está actualmente.



NOW WE ARE IN
THE SECOND CALL
TO fact. X is 2

if x == 1:

FACT	
X	2
FACT	
X	3

THE TOPMOST FUNCTION
CALL IS THE CALL WE
ARE CURRENTLY IN

else:

FACT	
X	2
FACT	
X	3

NOTE: BOTH FUNCTION CALLS
HAVE A VARIABLE NAMED X
AND THE VALUE OF X
IS DIFFERENT IN BOTH

return x * fact(x-1)

FACT	
X	1
FACT	
X	2
FACT	
X	3

YOU CAN'T ACCESS
THIS CALL'S X
FROM THIS CALL
AND VICE VERSA

if x == 1:

FACT	
X	1
FACT	
X	2
FACT	
X	3

WOW, WE MADE
THREE CALLS TO
fact, BUT WE
HAD NOT FINISHED
A SINGLE CALL UNTIL
NOW!

return 1

FACT	
X	1
FACT	
X	2
FACT	
X	3

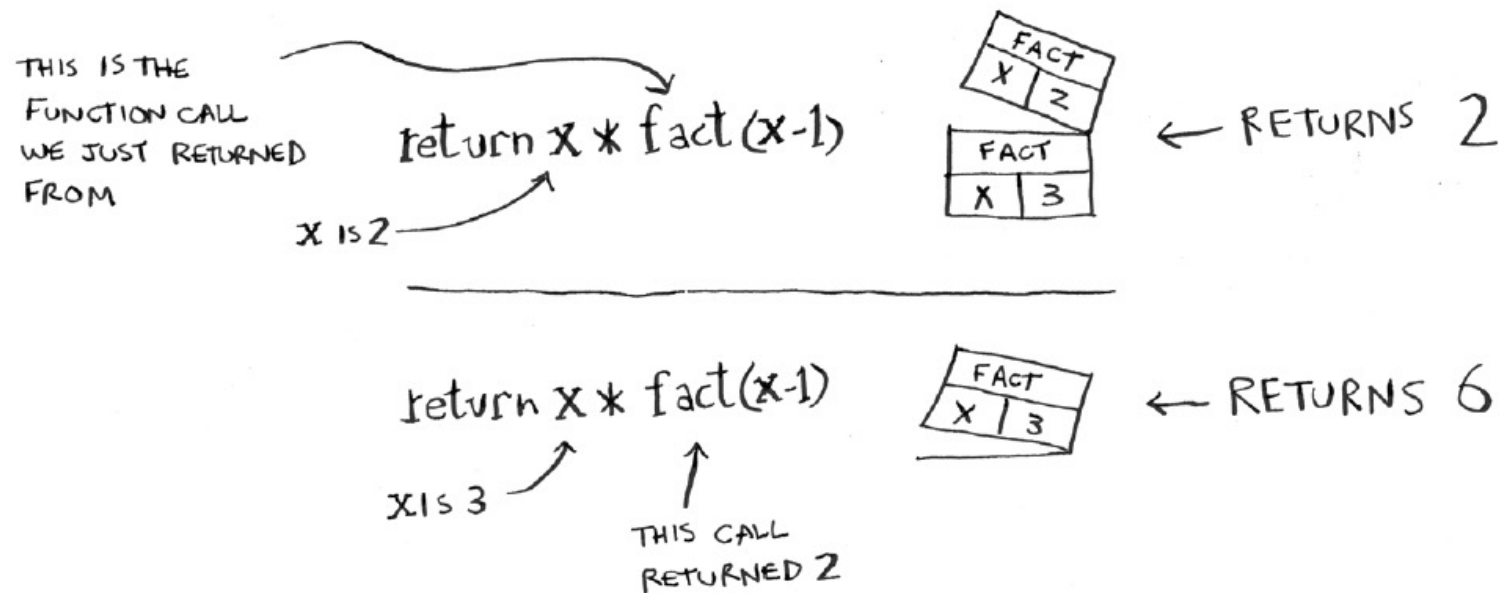
THIS IS THE FIRST BOX
TO GET POPPED OFF THE
STACK, WHICH MEANS
ITS THE FIRST CALL WE
RETURN FROM

RETURNS 1

```
def fact(x):
    if x == 1:
        return 1
    else:
        return x * fact(x-1)
```

La pila de llamadas con recursividad

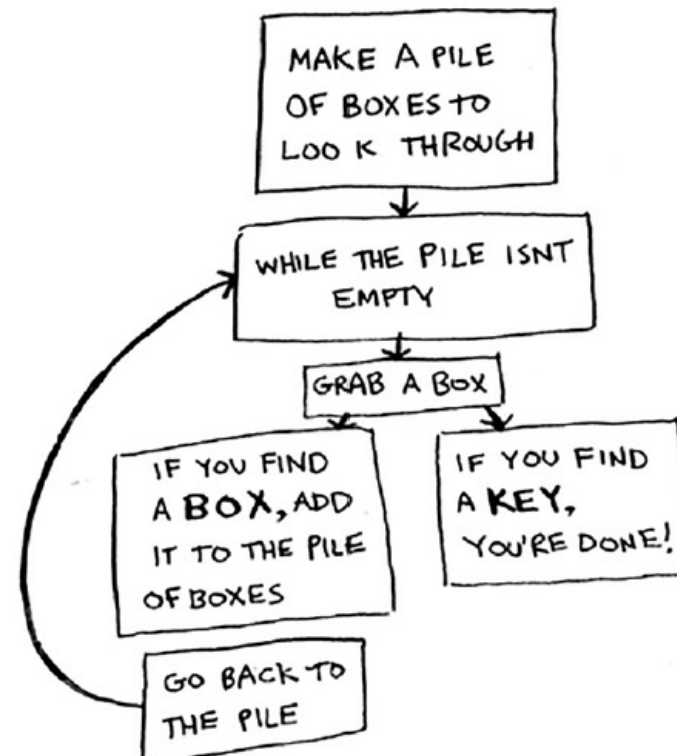
```
def fact(x):  
    if x == 1:  
        return 1  
    else:  
        return x * fact(x-1)
```



- Tenga en cuenta que cada llamada a `fact` tiene su propia copia de `x`. No puede acceder a una copia de `x` de una función diferente.

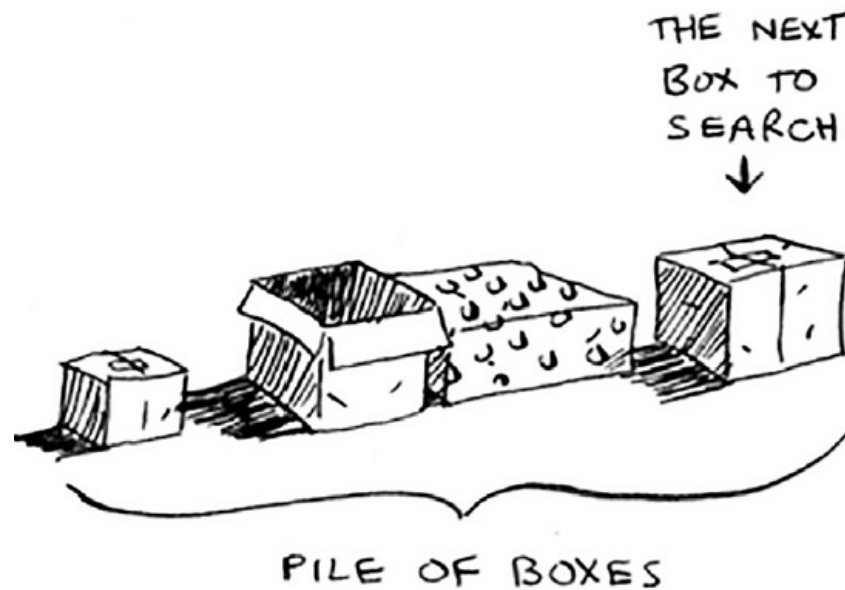
La pila de llamadas con recursividad

- La pila juega un papel importante en la recursividad. En el ejemplo inicial, hubo dos enfoques para encontrar la llave. Aquí está la primera forma de nuevo.



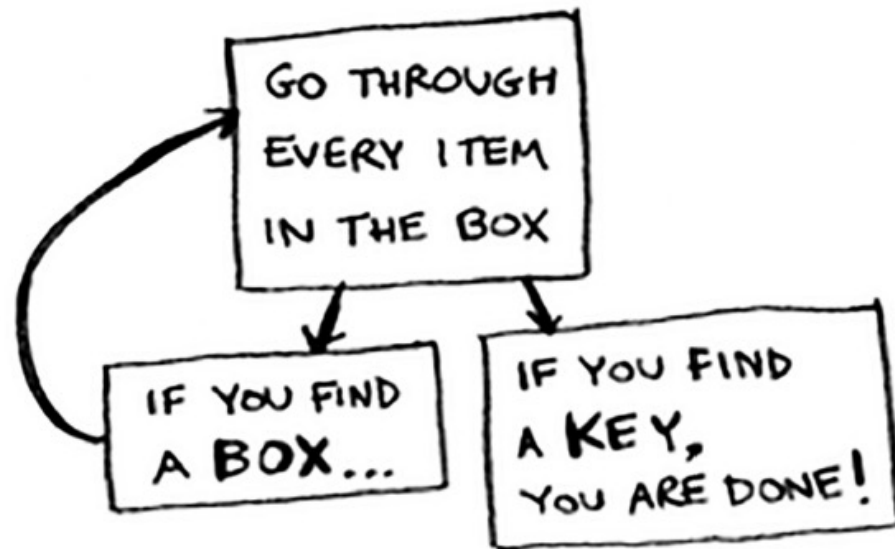
La pila de llamadas con recursividad

- De esta manera, crea una pila de cajas para buscar, de modo que siempre sepa qué cajas aún necesita buscar.



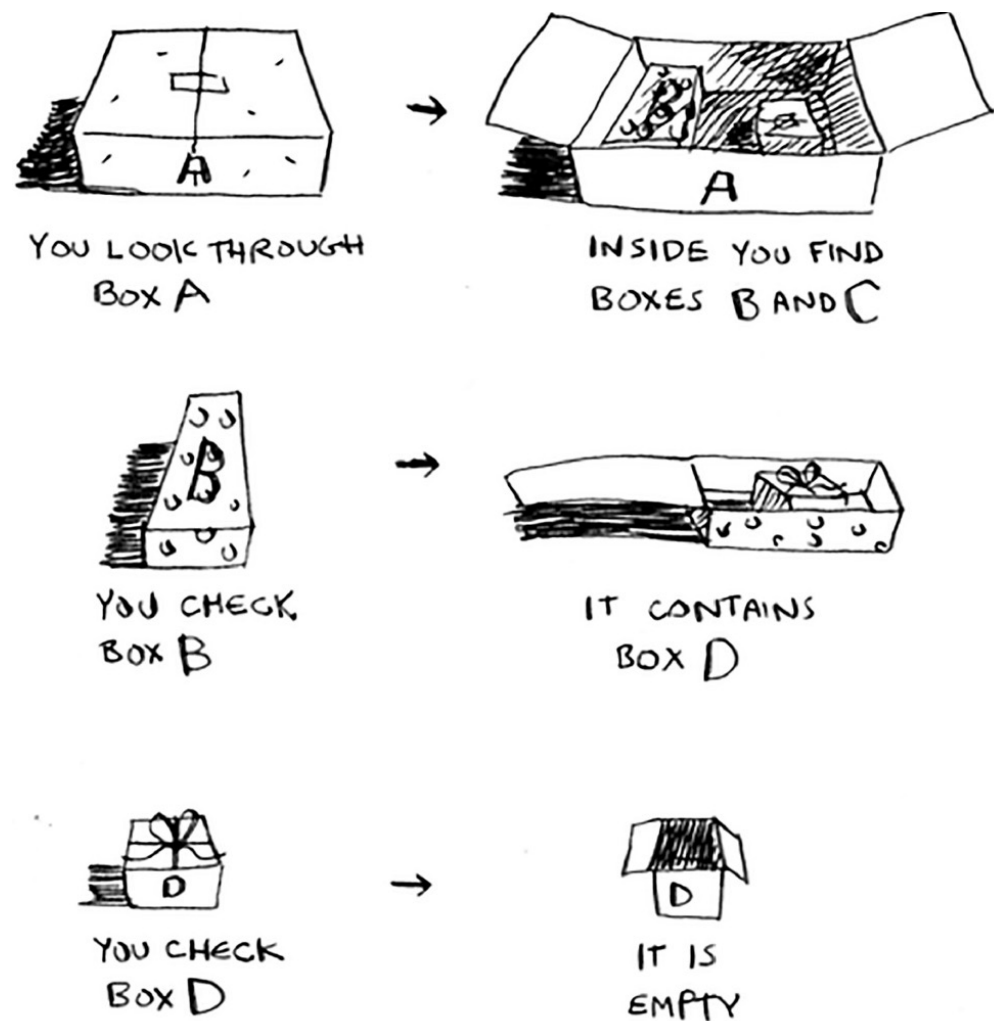
La pila de llamadas con recursividad

- Pero en el enfoque recursivo, no hay una estructura de pila.



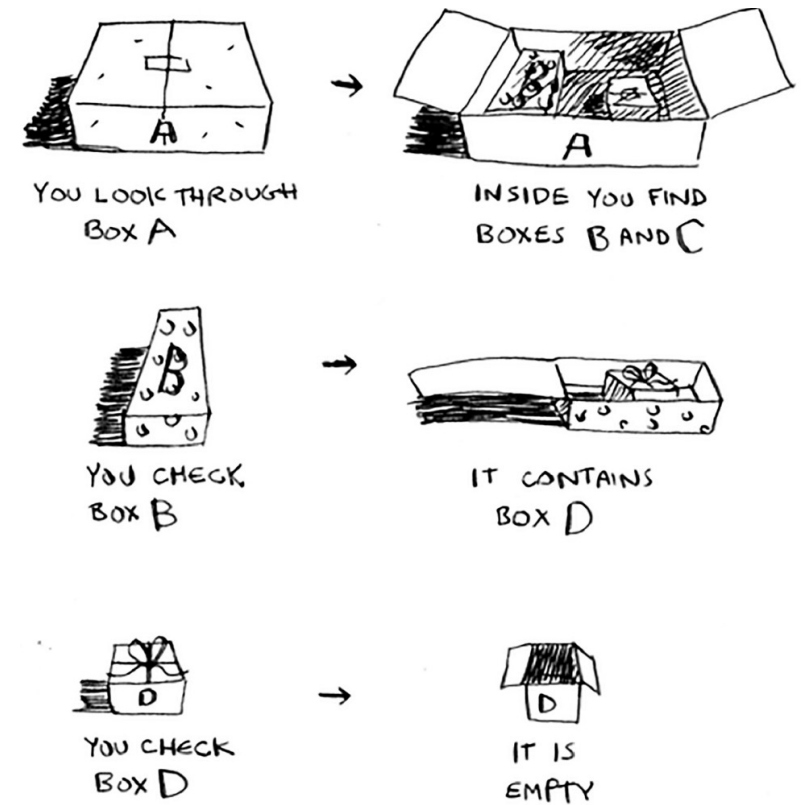
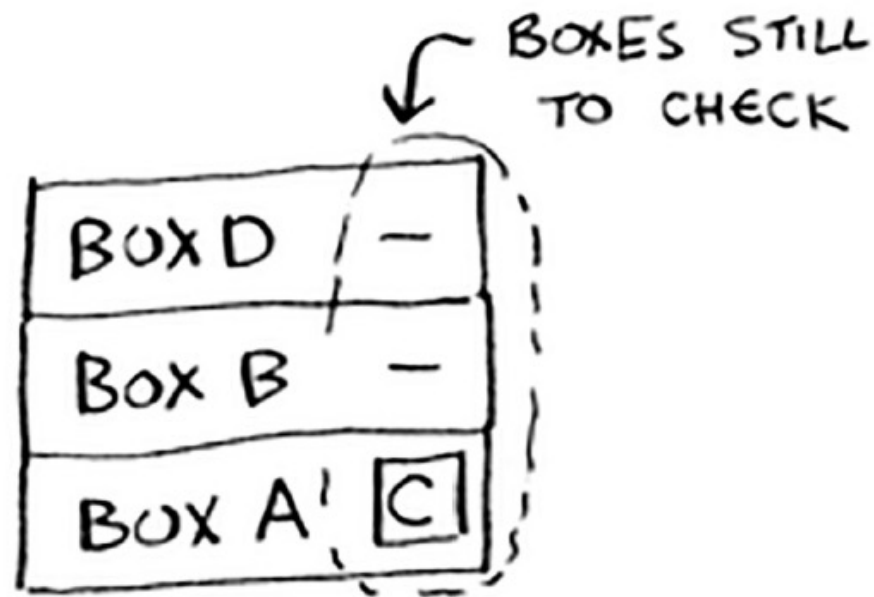
La pila de llamadas con recursividad

- Si no hay una pila, ¿cómo sabe su algoritmo qué cajas todavía tiene que mirar? Aquí hay un ejemplo.



La pila de llamadas con recursividad

- En este punto, la **pila de llamadas** se ve así.



La pila de llamadas con recursividad

- ¡El "montón de cajas" se guarda en la pila de llamadas!
- Esta es una **pila de llamadas de función a medio completar**, cada una con su propia lista de cajas a medio completar para examinar. El uso de la pila es conveniente porque *no tiene que hacer un seguimiento de una pila de cajas, la pila de llamadas lo hace por usted*.
- Usar la pila de llamadas es conveniente, pero tiene un costo: guardar toda esa información puede ocupar mucha memoria. Cada una de esas llamadas a funciones ocupa algo de memoria, y cuando su pila es demasiado alta, eso significa que su computadora está guardando información para muchas llamadas a funciones.
- En ese punto, **puede reescribir su código para usar un bucle en su lugar**.

La pila de llamadas con recursividad

EXERCISE

- 3.2** Suppose you accidentally write a recursive function that runs forever. As you saw, your computer allocates memory on the stack for each function call. What happens to the stack when your recursive function runs forever?

La pila de llamadas con recursividad

Answer: The stack grows forever. Each program has a limited amount of space on the call stack. When your program runs out of space (which it eventually will), it will exit with a stack-overflow error.